BITS Pilani

Machine Learning Operations (MLOps) · AIMLCZG523

Heart Disease Risk Prediction

End-to-End MLOps Pipeline

Assignment 01 · Project Report · Total marks 50

Field	Details
Student	JONNALAGADDA NAGA VENKATA SAI NIKHIL
Student ID	2025ae05575
Course	Machine Learning Operations (MLOps), AIMLCZG523
Dataset	Heart Disease UCI (Cleveland), 303 patients, 13 features
Code repository	https://github.com/NIK183/heart-disease-mlops-pipeline
Tech stack	scikit-learn · MLflow · FastAPI · Docker · Kubernetes/Helm · GitHub Actions · Prometheus · Grafana

Table of Contents

1. Introduction and Objectives 3
1.1 Summary of deliverables 3
1.2 Technology choices 4
2. Setup and Installation 4
3. Data Acquisition and Exploratory Data Analysis 5
3.1 Dataset and acquisition 5
3.2 Cleaning and preprocessing 5
3.3 Exploratory analysis 5
4. Feature Engineering and Model Development 7
4.1 Preprocessing pipeline 7
4.2 Candidate models and tuning 8
4.3 Results 8
4.4 Model selection 9
5. Experiment Tracking with MLflow 10
6. Model Packaging and Reproducibility 11
7. CI/CD Pipeline and Automated Testing 12
7.1 Unit tests 12
7.2 GitHub Actions workflow 12
8. Model Containerisation 13
9. Production Deployment (Kubernetes) 16
10. Monitoring and Logging 17
10.1 Request logging 17
10.2 Prometheus and Grafana 17
11. Architecture 19
11.1 Repository layout 20
12. Conclusion, Limitations and Future Work 20
Limitations 21
Future work 21
References and links 21

1. Introduction and Objectives

Cardiovascular disease is a leading cause of death worldwide, and early risk screening from routine clinical measurements can help prioritise patients for further diagnostic work-up. This project builds a binary classifier that predicts the presence of heart disease from 13 clinical attributes and delivers it the way a production ML service would be delivered: reproducible training, tracked experiments, automated testing, a containerised API, Kubernetes deployment and live monitoring.

The report follows the nine tasks of the assignment brief. Each section states what was built, the design decisions behind it, and the evidence (results, logs and screenshots) that it works.

Key results at a glance

Area	Outcome
Champion model	Logistic Regression (C = 0.1, L2), selected on nested-CV ROC-AUC
Cross-validated performance	ROC-AUC 0.903 (SD 0.015); accuracy 0.839
Held-out test (61 patients)	Accuracy 0.885; recall 0.929; ROC-AUC 0.967
Quality gates	21 pytest tests, flake8, three-job GitHub Actions pipeline passing
Serving and deployment	FastAPI in a non-root Docker image; Kubernetes with 2 replicas and HPA (2 to 5)
Monitoring	Prometheus and Grafana dashboard; p95 latency 24.8 ms under sample traffic

1.1 Summary of deliverables

Assignment task	Implementation in this repository
1. Data & EDA	src/data_loader.py, src/preprocessing.py, src/eda.py; notebooks/01_exploratory_data_analysis.ipynb
2. Features & models	Scikit-learn Pipeline; Logistic Regression, Random Forest, Gradient Boosting; nested cross-validation (src/train.py)
3. Experiment tracking	MLflow with SQLite backend; experiment "heart-disease-prediction"
4. Packaging	models/heart_disease_pipeline.joblib + model_metadata.json; pinned requirements.txt and environment.yml
5. CI/CD & testing	21 pytest tests; flake8; .github/workflows/ci.yml (3 jobs); Jenkinsfile as alternative
6. Containerisation	Dockerfile (python:3.12-slim, non-root, HEALTHCHECK); FastAPI /predict
7. Deployment	k8s/ manifests (Deployment, Service, Ingress, HPA) and Helm chart on Docker Desktop Kubernetes
8. Monitoring	Structured request logs; Prometheus + Grafana (monitoring/ docker-compose stack)
9. Documentation	This report, README.md, screenshot folder reports/screenshots/

1.2 Technology choices

- **scikit-learn Pipeline:** a single object holds imputation, scaling, encoding and the classifier, so training and serving cannot drift apart.
- **MLflow (SQLite backend):** lightweight, no server needed for local experiments, and gives a comparison UI for runs.
- **FastAPI + Pydantic:** automatic request validation, OpenAPI/Swagger docs and good async performance.
- **Docker Desktop Kubernetes + Helm:** a local cluster that behaves like GKE/EKS/AKS without cloud cost; the same manifests port to a managed cluster.
- **Prometheus + Grafana:** the standard open-source pairing for API metrics, with the dashboard provisioned as code.

2. Setup and Installation

All scripts run from a clean checkout using the pinned requirement files. Python 3.12 is required so the scikit-learn version used for training matches the version inside the Docker image (avoids model-serialisation skew).

```
git clone https://github.com/NIK183/heart-disease-mlops-pipeline
cd heart-disease-mlops-pipeline
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1          # Windows PowerShell
python -m pip install --upgrade pip
pip install -r requirements-dev.txt
```

A Makefile (and an equivalent Make.ps1 for Windows without GNU Make) wraps the common tasks:

Command	What it does
make data	Download (or fall back to the local copy of) the UCI data, clean it, generate EDA figures
make train	Tune three models with nested CV, log runs to MLflow, save the champion model
make lint / make test	Run strict flake8 and the pytest suite
make serve	Start the FastAPI server locally on port 8000
make mlflow-ui	Open the MLflow UI at http://127.0.0.1:5000
make docker-build / docker-run / docker-test	Build the image, run the container, smoke-test the endpoints
make monitoring-up / traffic	Start Prometheus and Grafana, then generate sample API traffic

Calling the API once the server is running:

```
curl -X POST http://127.0.0.1:8000/predict \
  -H "Content-Type: application/json" -d @sample_patient.json
```

3. Data Acquisition and Exploratory Data Analysis

3.1 Dataset and acquisition

The project uses the processed Cleveland subset of the UCI Heart Disease dataset: 303 patient records, 13 predictors and a diagnosis column that takes values 0 to 4. src/data_loader.py downloads the file from the UCI repository and, if the download fails (offline use, UCI outage), falls back to the copy shipped in dataset/. The raw file is written to data/raw/ and the cleaned file to data/processed/, so the pipeline is repeatable without internet access.

Feature	Type	Meaning
age	numeric	Age in years
sex	binary	1 = male, 0 = female
cp	categorical	Chest pain type (1 typical angina, 2 atypical, 3 non-anginal, 4 asymptomatic)
trestbps	numeric	Resting blood pressure (mm Hg)
chol	numeric	Serum cholesterol (mg/dl)
fbs	binary	Fasting blood sugar above 120 mg/dl
restecg	categorical	Resting ECG result (0 normal, 1 ST-T abnormality, 2 LV hypertrophy)
thalach	numeric	Maximum heart rate achieved
exang	binary	Exercise-induced angina
oldpeak	numeric	ST depression induced by exercise relative to rest
slope	categorical	Slope of the peak exercise ST segment
ca	categorical	Major vessels (0 to 3) coloured by fluoroscopy
thal	categorical	Thalassemia (3 normal, 6 fixed defect, 7 reversible defect)

3.2 Cleaning and preprocessing

- Missing values are encoded as "?" in the source. They are converted to NaN and every feature column is cast to numeric. Only 6 values are missing: 4 in ca and 2 in thal.
- The original 5-level diagnosis (0 to 4) is binarised: 0 = no disease (164 patients), 1 to 4 = disease present (139 patients).
- Imputation, scaling and encoding are not applied to the saved CSV; they live inside the sklearn Pipeline (section 4) so that they are fitted only on training folds. This prevents data leakage.

3.3 Exploratory analysis

The classes are close to balanced (54% healthy, 46% disease), so accuracy is a reasonable metric and no resampling is needed. Recall is still tracked separately because missing a diseased patient is the costlier error in a screening setting.

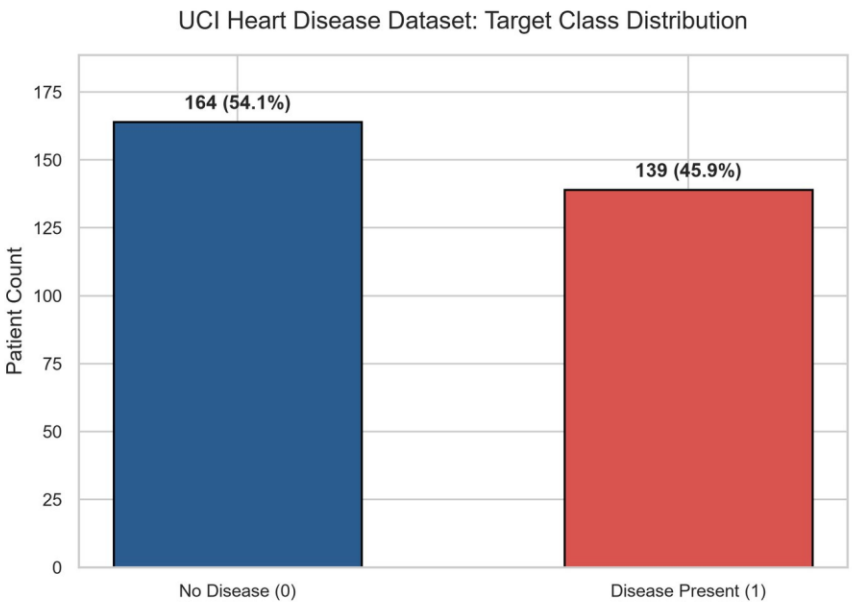


Figure 1: Class balance of the binary target (164 no disease vs 139 disease).

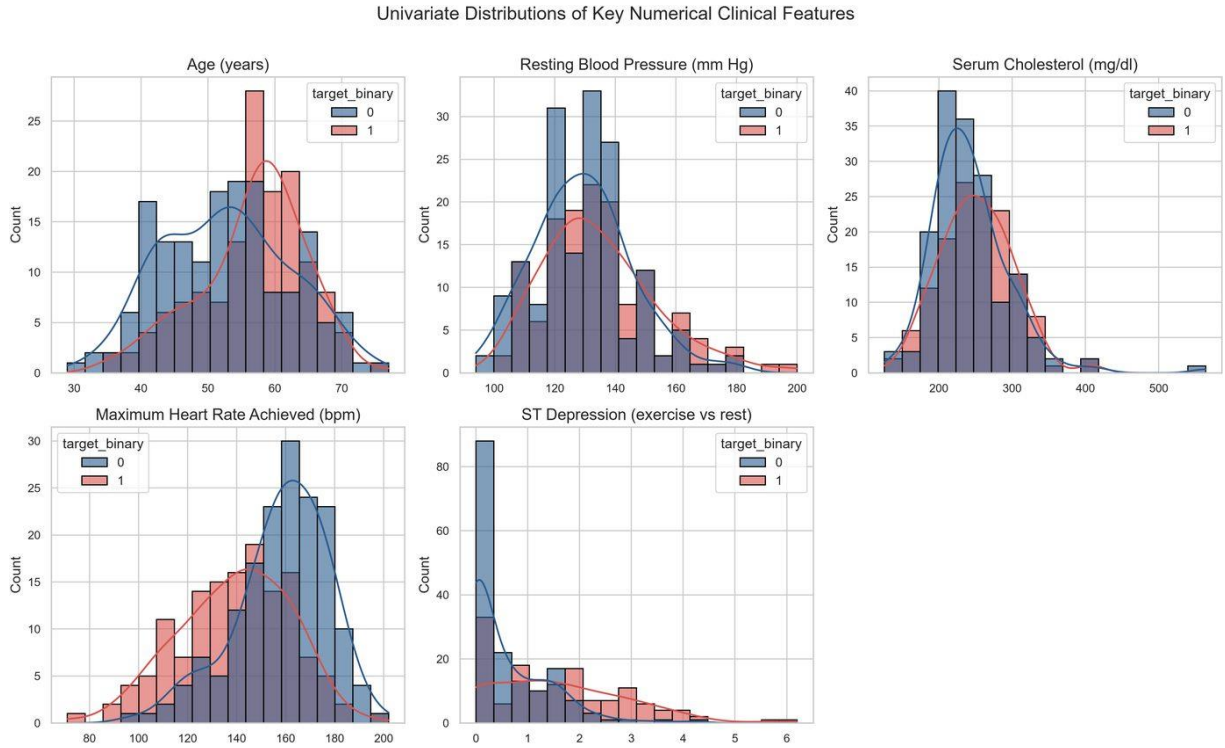


Figure 2: Distributions of the clinical features, split by diagnosis.

The distributions show clear separation for several predictors: patients with disease tend to have lower maximum heart rate (thalach), higher ST depression (oldpeak), and more often present with asymptomatic chest pain, exercise-induced angina and a higher vessel count (ca).

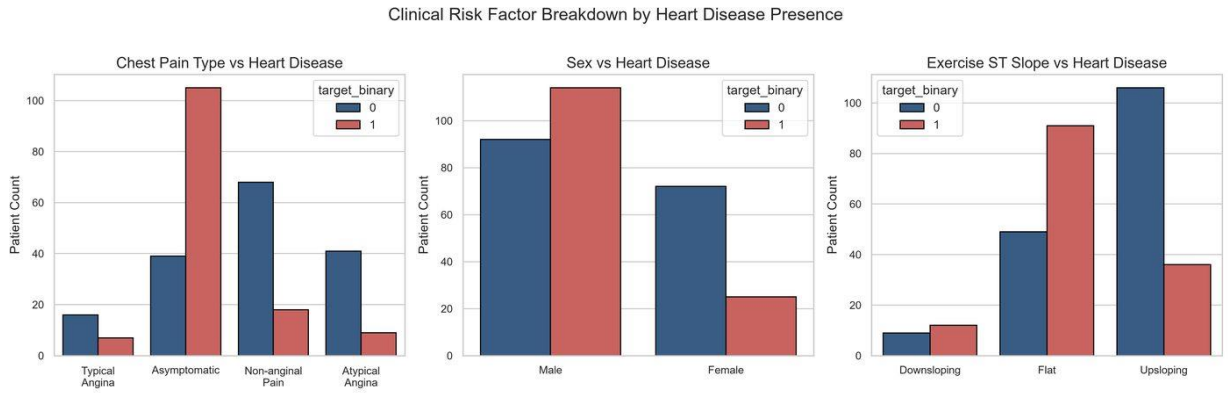


Figure 3: Clinical comparisons between patients with and without heart disease.

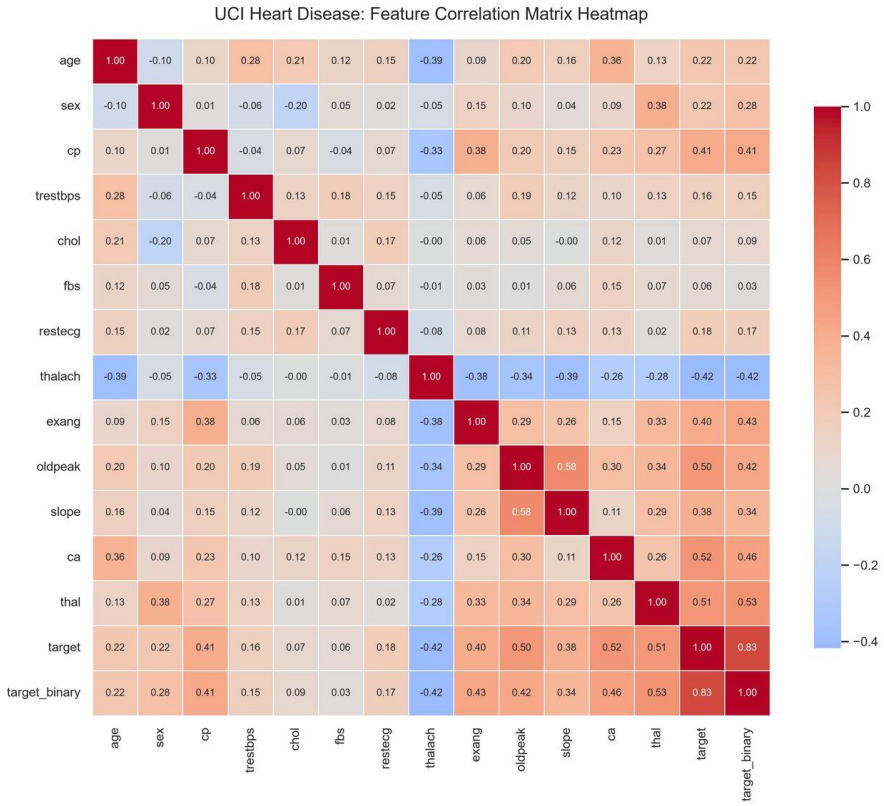


Figure 4: Correlation heatmap of the features and target.

In the correlation heatmap, thal, ca, oldpeak, exang and cp show the strongest positive association with the target, while thalach is negatively associated. No pair of predictors is so strongly correlated that it would make a linear model unstable.

4. Feature Engineering and Model Development

4.1 Preprocessing pipeline

Features are split by type and handled in a ColumnTransformer that is bundled with the classifier into a single Pipeline:

- **Numerical (age, trestbps, chol, thalach, oldpeak):** median imputation, then standard scaling.
- **Categorical (sex, cp, fbs, restecg, exang, slope, ca, thal):** most-frequent imputation, then one-hot encoding with handle_unknown="ignore" so unseen categories at inference time do not crash the API.

4.2 Candidate models and tuning

Three classifiers were trained, which exceeds the minimum of two. Each is tuned with GridSearchCV (3-fold inner CV, scoring ROC-AUC) on an 80% stratified training split; a stratified 5-fold outer CV then estimates generalisation (nested cross-validation). The remaining 20% (61 patients) is a held-out test set that is evaluated once and never used for selection.

Model	Grid size	Search space	Best parameters
Logistic Regression	28	C: 0.01 to 10; penalty: l1/l2; class_weight: none/balanced	C = 0.1, penalty = l2, class_weight = none
Random Forest	24	n_estimators: 100/200; max_depth: 3/5/8/None; min_samples_leaf: 1/2/4	200 trees, max_depth = 3, min_samples_leaf = 2
Gradient Boosting	12	n_estimators: 100/200; learning_rate: 0.03/0.05/0.1; max_depth: 2/3	100 trees, learning_rate = 0.03, max_depth = 2

4.3 Results

Results as logged in MLflow (nested 5-fold CV on the training split, plus the single held-out test evaluation):

Model	CV Acc.	CV ROC-AUC	Test Acc.	Test Prec.	Test Recall	Test ROC-AUC
Logistic Regression	0.839	0.903	0.885	0.839	0.929	0.967
Random Forest	0.810	0.890	0.836	0.800	0.857	0.943
Gradient Boosting	0.818	0.878	0.902	0.867	0.929	0.954

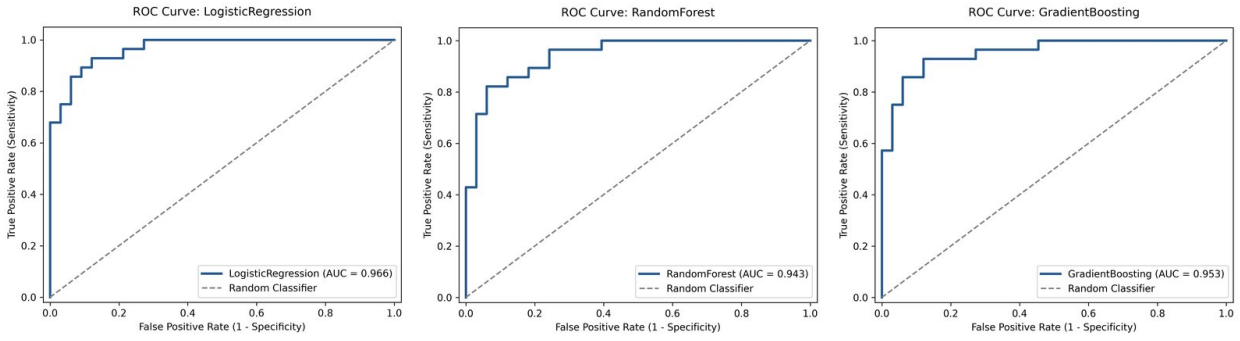


Figure 5: ROC curves on the held-out test set: Logistic Regression, Random Forest, Gradient Boosting.

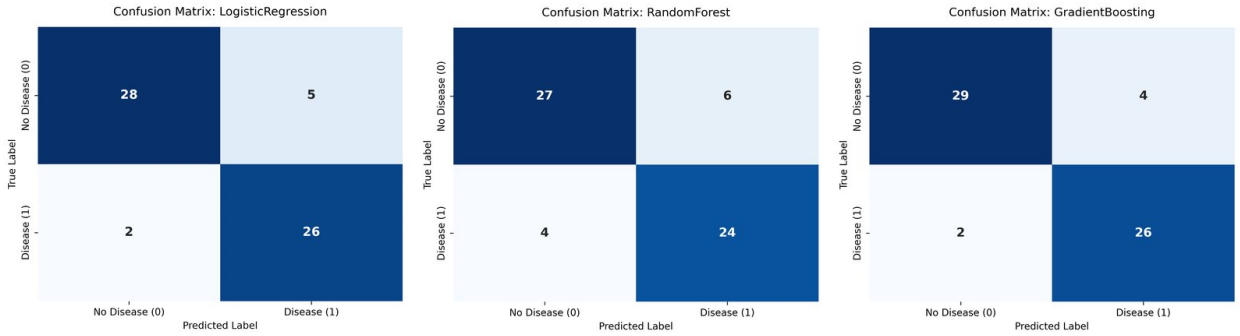


Figure 6: Confusion matrices on the held-out test set (same model order).

4.4 Model selection

The champion is chosen by highest nested-CV ROC-AUC, with nested-CV accuracy as tie-breaker. Logistic Regression wins on both (0.903 AUC, 0.839 accuracy). Gradient Boosting happens to score slightly higher accuracy on the 61-patient test split (0.902 vs 0.885), but with only 61 samples that gap is about one patient, and basing the choice on the test set would bias the estimate. Choosing on cross-validation keeps the test set an honest, unseen check.

Logistic Regression is also the best fit for the use case: it is the most stable across folds (AUC standard deviation 0.015 vs 0.032 to 0.035 for the tree ensembles), it is fast to serve, and its coefficients are directly interpretable by clinicians.

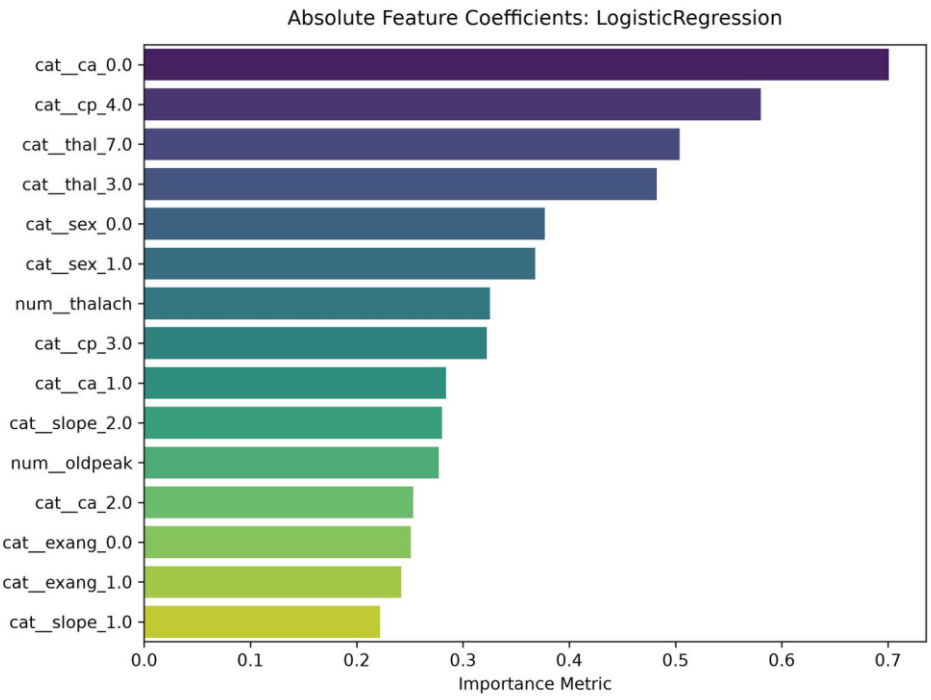


Figure 7: Feature importance of the champion Logistic Regression model.

5. Experiment Tracking with MLflow

Every training run is logged to MLflow with a SQLite backend (mlflow.db) under the experiment "heart-disease-prediction". One run is created per candidate model and records:

- Parameters: baseline settings, best tuned hyper-parameters, grid size, outer/inner CV folds, test size and random seed.
- Metrics: best inner-CV ROC-AUC; nested-CV mean and standard deviation of accuracy, precision, recall, F1 and ROC-AUC; held-out test metrics.
- Artifacts: confusion matrix, ROC curve, feature-importance plot, classification report, full grid-search results CSV and the fitted sklearn pipeline with an input example.

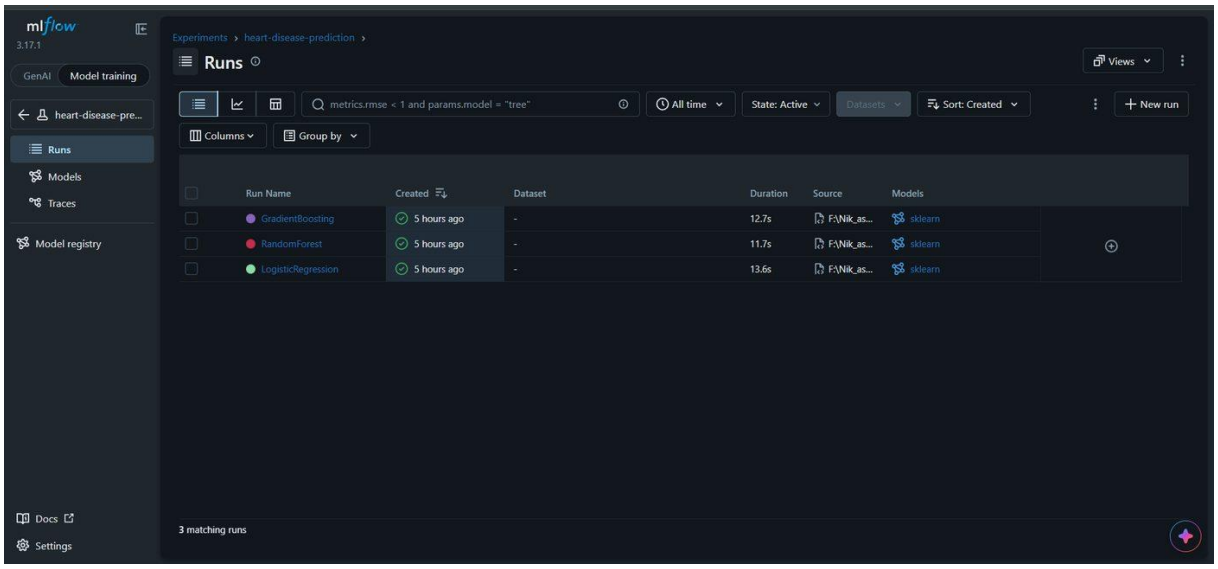


Figure 8: MLflow experiment view listing the three tracked runs.

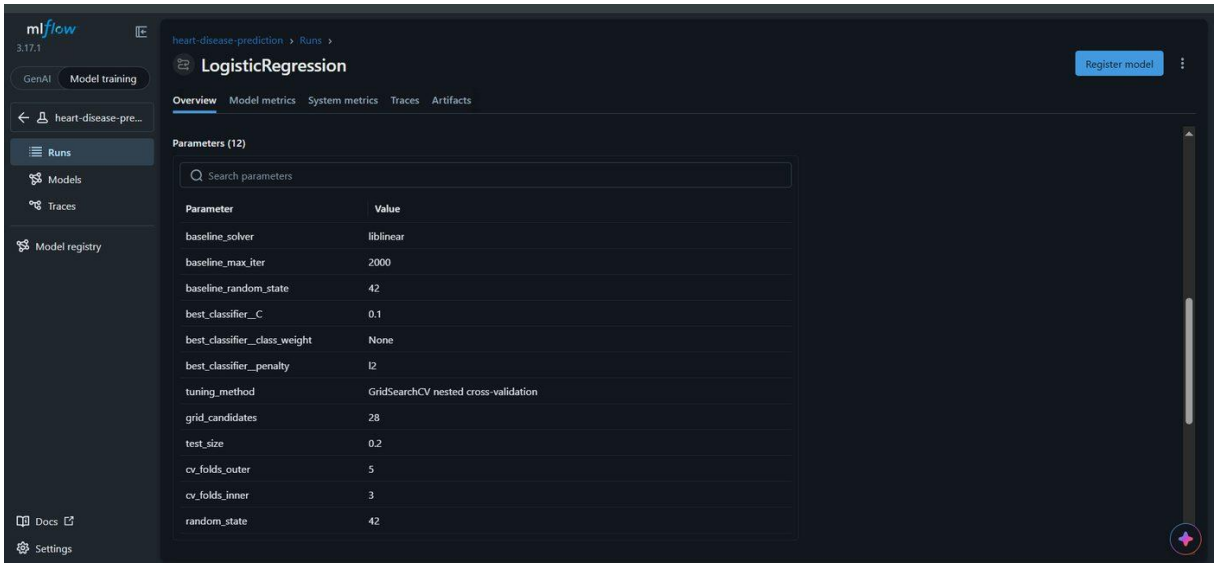


Figure 9: Parameters logged for a run.

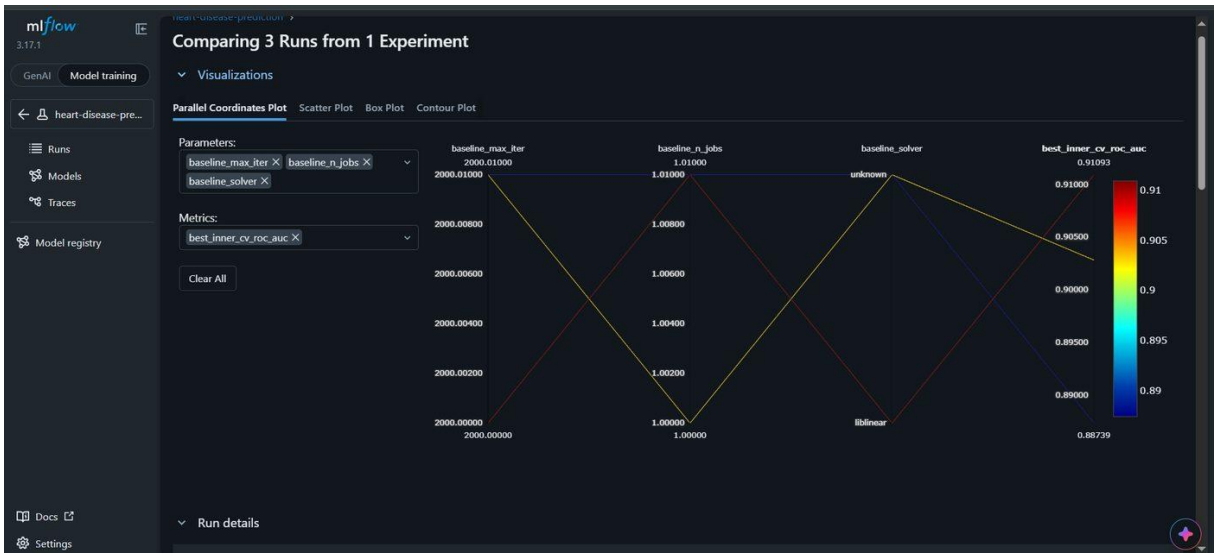


Figure 10: Side-by-side comparison of the three runs in MLflow.

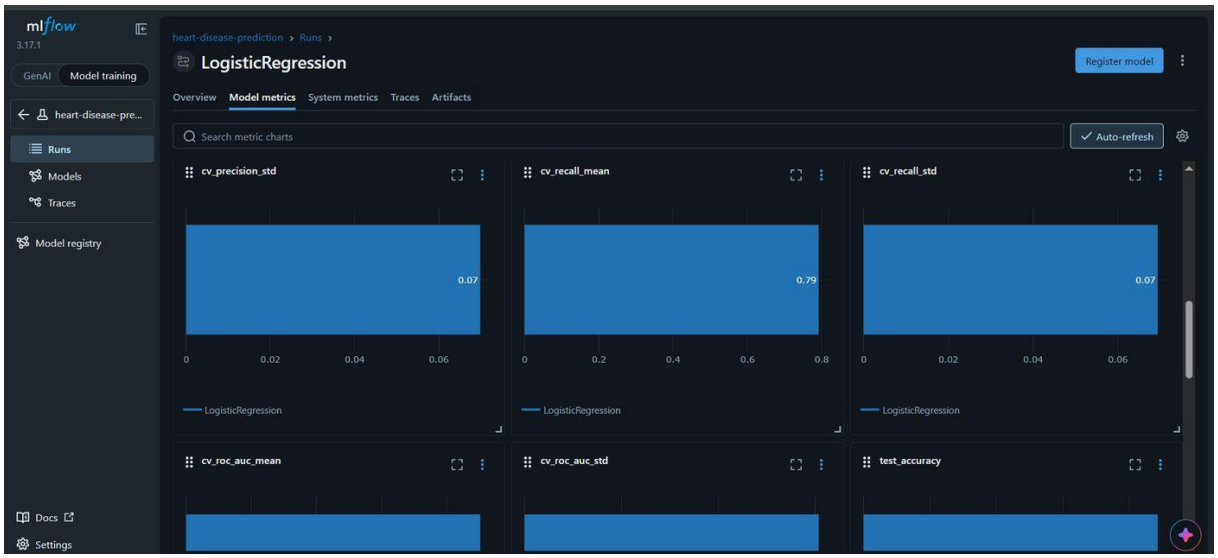


Figure 11: Cross-validated ROC-AUC across models (MLflow metric chart).

6. Model Packaging and Reproducibility

- **Model artifact:** the fitted Pipeline (preprocessing + classifier) is saved with joblib as models/heart_disease_pipeline.joblib. The same pipeline is also stored as an MLflow model.
- **Metadata:** models/model_metadata.json records the champion name, tuned parameters, selection criterion and the ordered feature list.
- **Dependencies:** requirements.txt pins runtime packages (e.g. scikit-learn 1.6.1, pandas 2.2.3, numpy 2.2.3, fastapi 0.115.12); requirements-dev.txt adds pytest, flake8 and MLflow; environment.yml gives a Conda alternative.
- **Determinism:** random_state = 42 for splits, CV folds and models; the 80/20 split is stratified.
- **No training/serving skew:** because the transformers are part of the saved pipeline, the API sends raw clinical values and gets exactly the same preprocessing as in training.

7. CI/CD Pipeline and Automated Testing

7.1 Unit tests

The test/ folder holds 21 pytest tests in three modules, run with coverage:

Module	Tests	What is covered
test_data_and_preprocessing.py	7	"?" replaced by NaN; binary target created; imputation and scaling; one-hot output is binary; unseen category does not crash; feature schema; 303 rows
test_training.py	5	Metric computation on known values; every grid key valid for the Pipeline; grid sizes; saved model predicts 0/1; probabilities in [0,1]
test_api.py	9	Root and /health; valid /predict; missing field rejected (422); out-of-range cp rejected; determinism; /metrics exposes custom counters; /docs available; probabilities bounded

7.2 GitHub Actions workflow

.github/workflows/ci.yml runs on every push and pull request to main/master (and manually). It has three dependent jobs; a failure in any job fails the run and stops the later jobs:

Job	Steps
lint-and-test	Install requirements-dev.txt; run flake8 (max complexity 10, line length 127); run pytest with JUnit XML and coverage; upload logs and reports as an artifact
train-and-evaluate	Run data_loader, preprocessing, eda and train; verify that the model, metadata, comparison CSV, tuning CSV and mlflow.db exist; upload trained artifacts
docker-build-verify	Download trained artifacts; build the image; start the container; poll /health; POST a sample patient to /predict and assert a valid response; check /metrics; upload container logs

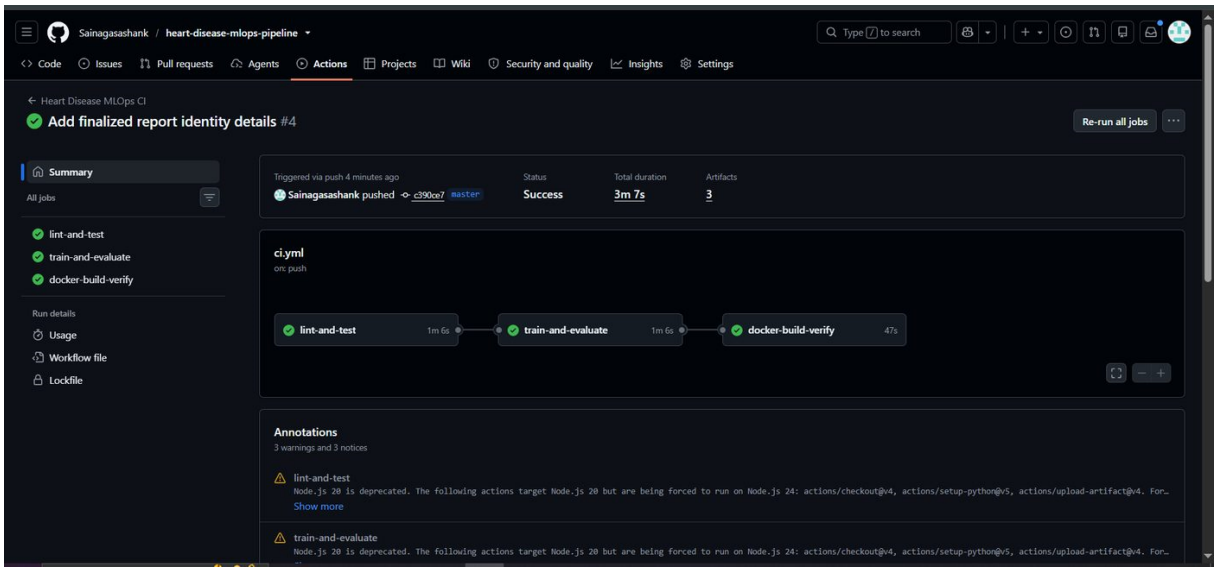


Figure 12: Successful GitHub Actions run: all three jobs passed (total 3m 7s, 3 artifacts).

A Jenkinsfile with the same stages (install, lint and test, train, Docker verify) is included as an alternative CI definition.

8. Model Containerisation

The serving API is a FastAPI application (src/api/app.py) with these endpoints:

Endpoint	Method	Purpose
/predict	POST	Accepts the 13 patient features as JSON and returns prediction (0/1), risk label, confidence, disease and healthy probabilities
/health	GET	Liveness/readiness probe; confirms the model is loaded
/metrics	GET	Prometheus metrics
/docs	GET	Interactive Swagger UI
/	GET	Service metadata

Input validation is handled by a Pydantic schema with clinical range limits (for example cp must be 1 to 4, age 1 to 120), so malformed requests are rejected with HTTP 422 before reaching the model.

The Dockerfile uses python:3.12-slim, installs the pinned requirements, copies only src/ and models/, runs as a non-root user and defines a HEALTHCHECK against /health. Uvicorn serves the app on port 8000.

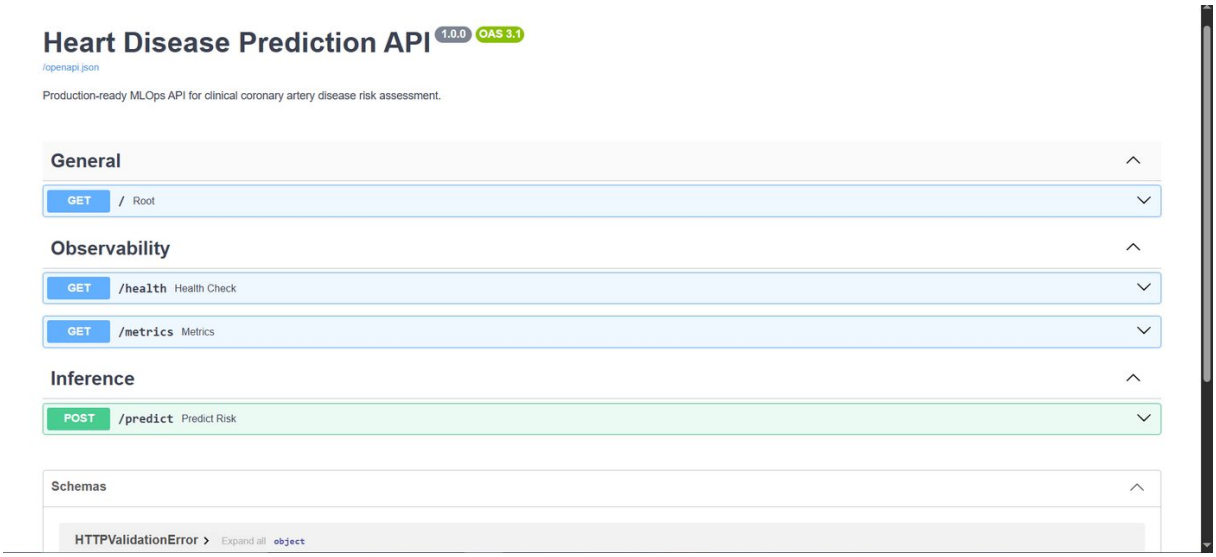


Figure 13: Swagger UI of the running API.

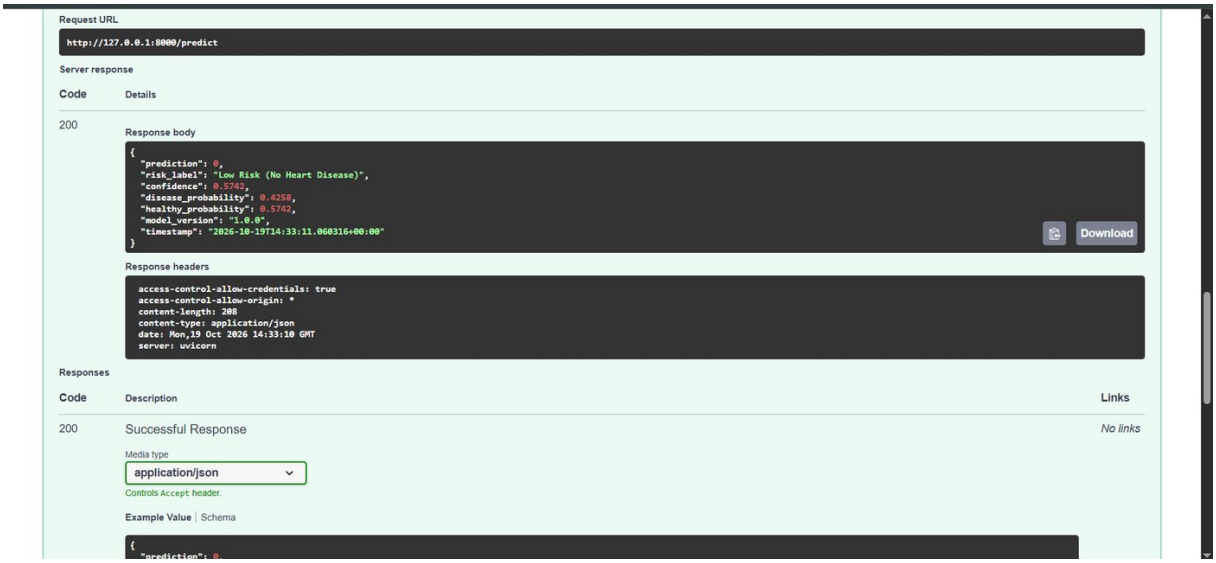


Figure 14: Example /predict request and response.

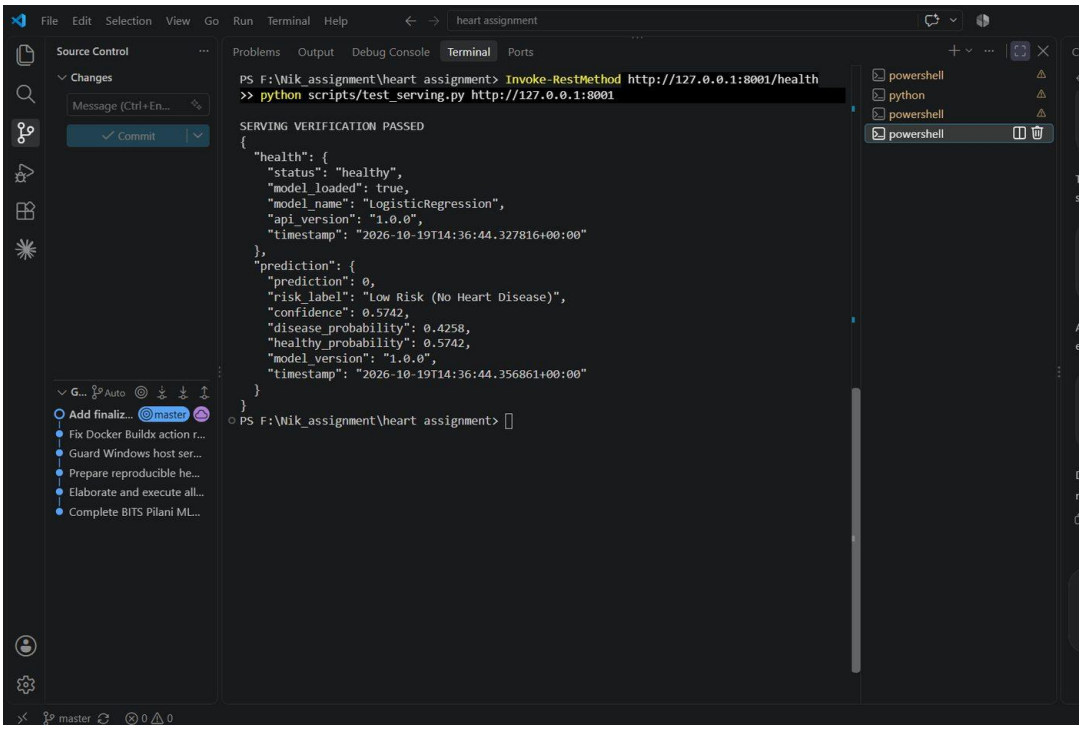


Figure 15: Docker image build and container run.

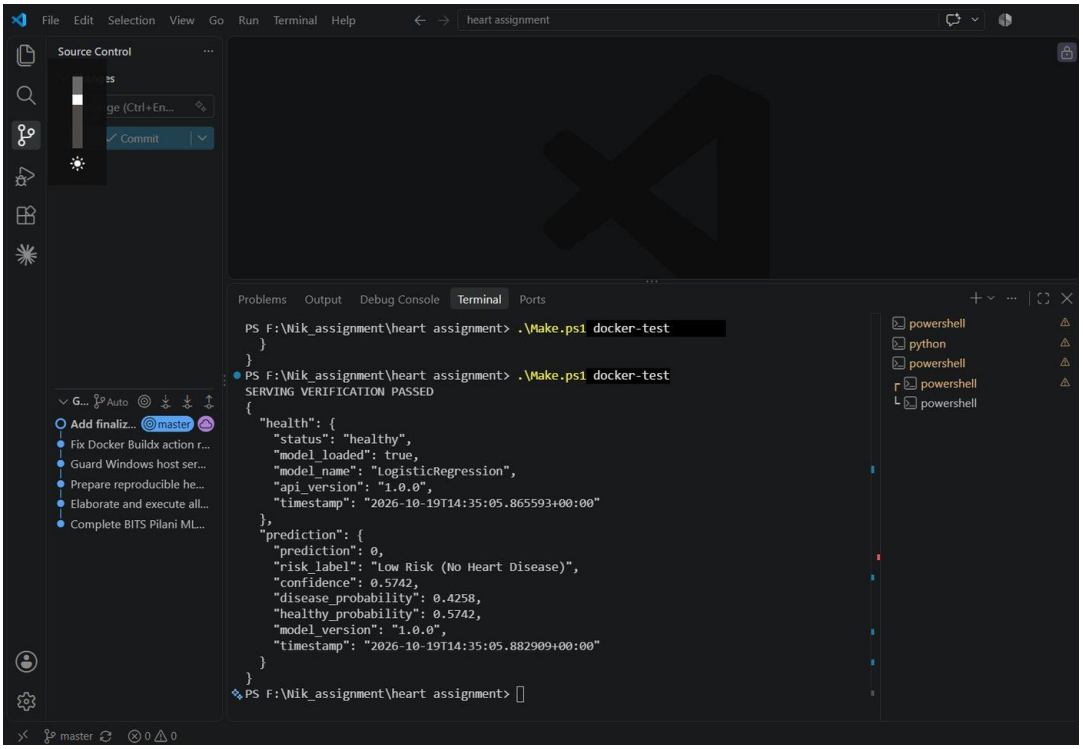


Figure 16: Serving verification passing (health check and a sample prediction: Low Risk, confidence 0.5742).

9. Production Deployment (Kubernetes)

The container is deployed to the Kubernetes cluster built into Docker Desktop using manifests in k8s/. An equivalent Helm chart is provided in helm/heart-disease-api/.

Resource	Configuration
Deployment	2 replicas; rolling update (maxSurge 1, maxUnavailable 0); CPU 100m/500m, memory 128Mi/512Mi requests/limits; liveness and readiness probes on /health
Service	Type LoadBalancer; port 80 to container port 8000
Ingress	Routes / to the service (nginx rewrite annotation)
HorizontalPodAutoscaler	2 to 5 replicas, target 70% CPU
Helm chart	Same resources, parameterised through values.yaml

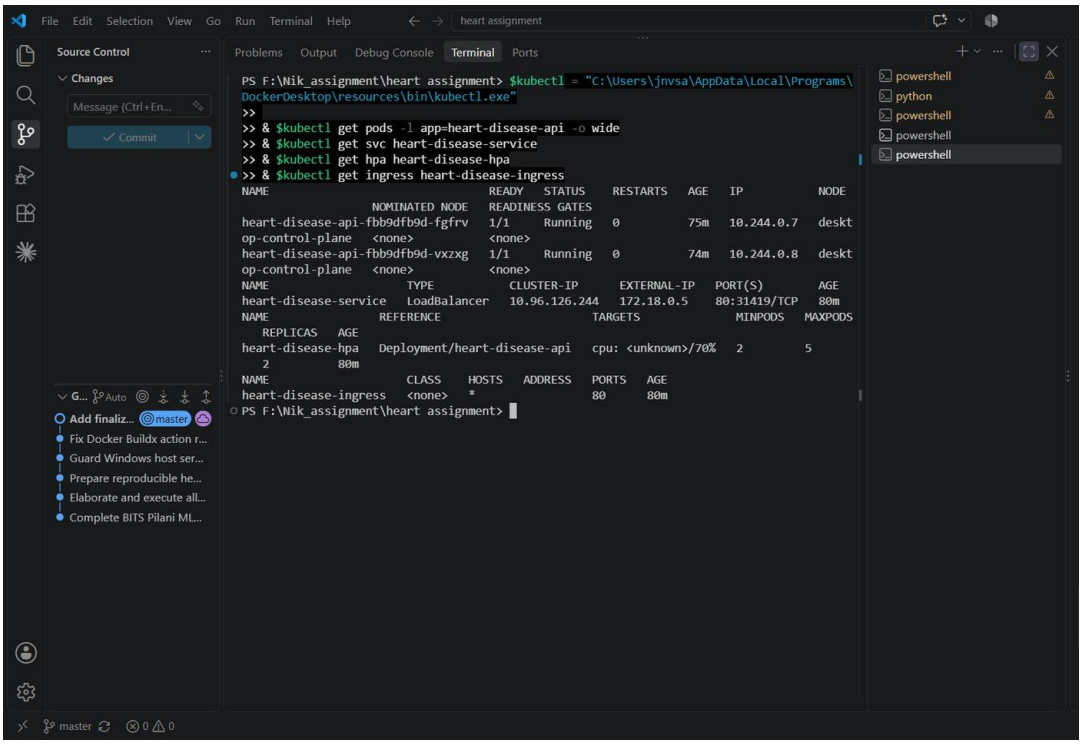


Figure 17: kubectl output: two API pods Running, LoadBalancer service, HPA and Ingress.

Both pods are Running with zero restarts and the service is reachable. The API is exposed through the LoadBalancer service and verified with scripts/test_serving.py against a port-forward to the service, using the same smoke test as in the Docker stage. The deployment is local (not on a public cloud), so access instructions are in the README rather than a public URL.

> Note: the HPA column shows "cpu: <unknown>/70%" in the screenshot. This is typical on Docker Desktop when the Kubernetes metrics-server add-on is not installed; the autoscaler object is configured correctly, but live CPU-based scaling would need metrics-server enabled.

10. Monitoring and Logging

10.1 Request logging

A FastAPI middleware logs every request as structured JSON (client IP, method, path, status code, duration in ms), and each prediction logs its outcome, confidence and latency. These logs are visible with docker logs or kubectl logs.

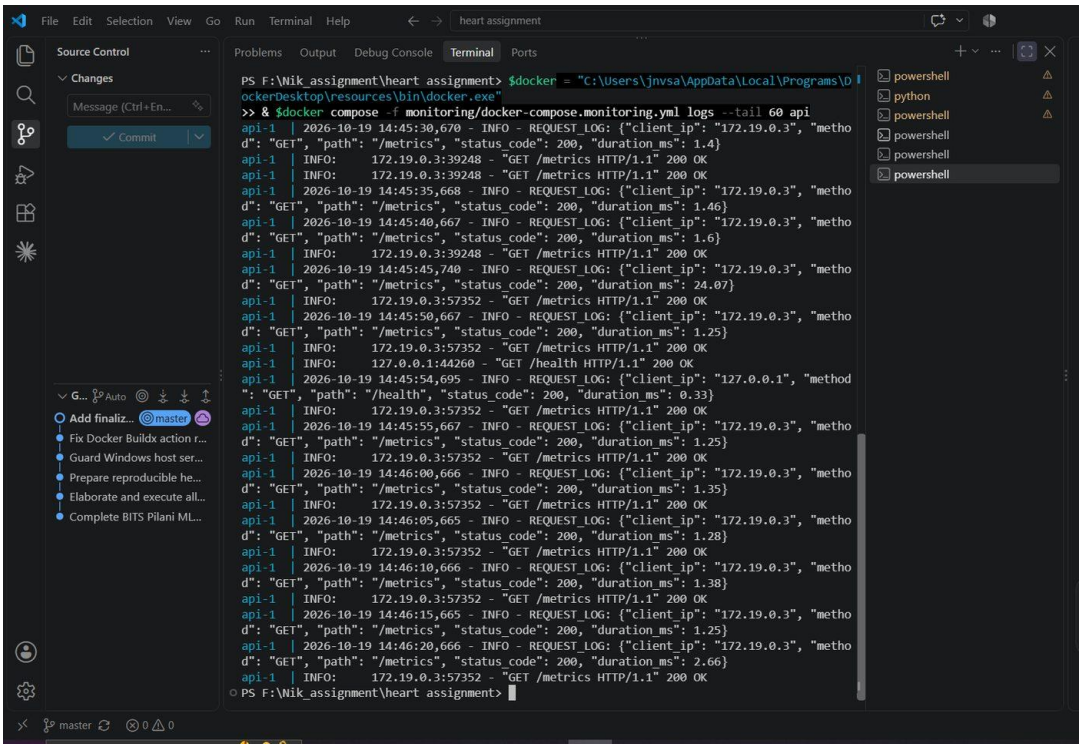


Figure 18: API request logs showing method, path, status code and latency.

10.2 Prometheus and Grafana

The API exposes standard HTTP metrics (via prometheus-fastapi-instrumentator) and two custom ones: heart_disease_predictions_total, labelled by outcome and risk label, and heart_disease_inference_latency_seconds, a latency histogram. Prometheus scrapes /metrics every 5 seconds, and Grafana is provisioned with the Prometheus data source and a "Heart Disease Model Serving Dashboard" (request rate, p95 latency, predictions by outcome, status codes, latency percentiles). The stack starts with a single docker compose command.

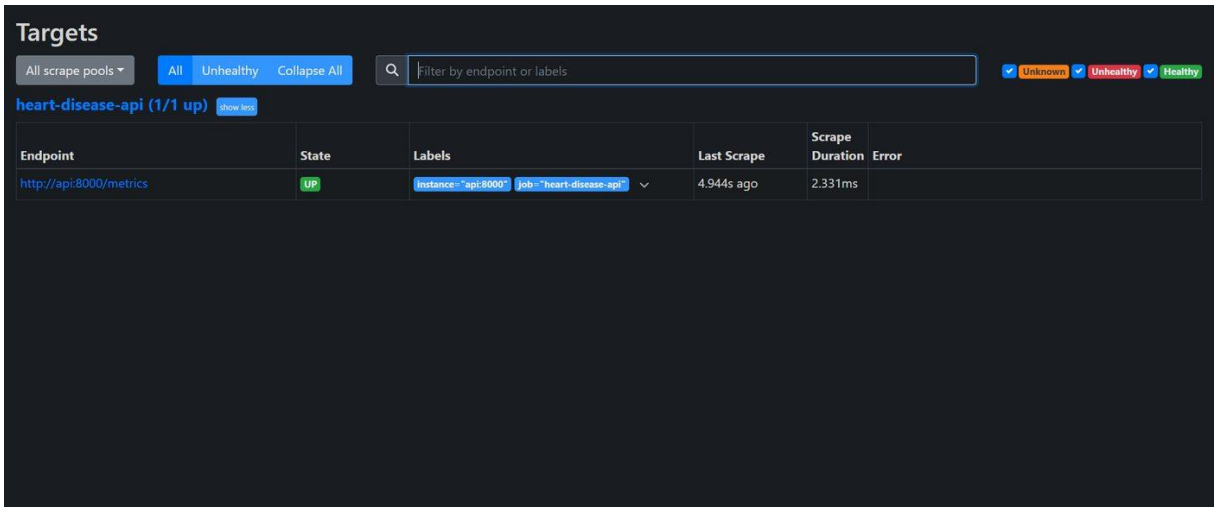


Figure 19: Prometheus target page: the API target is UP.

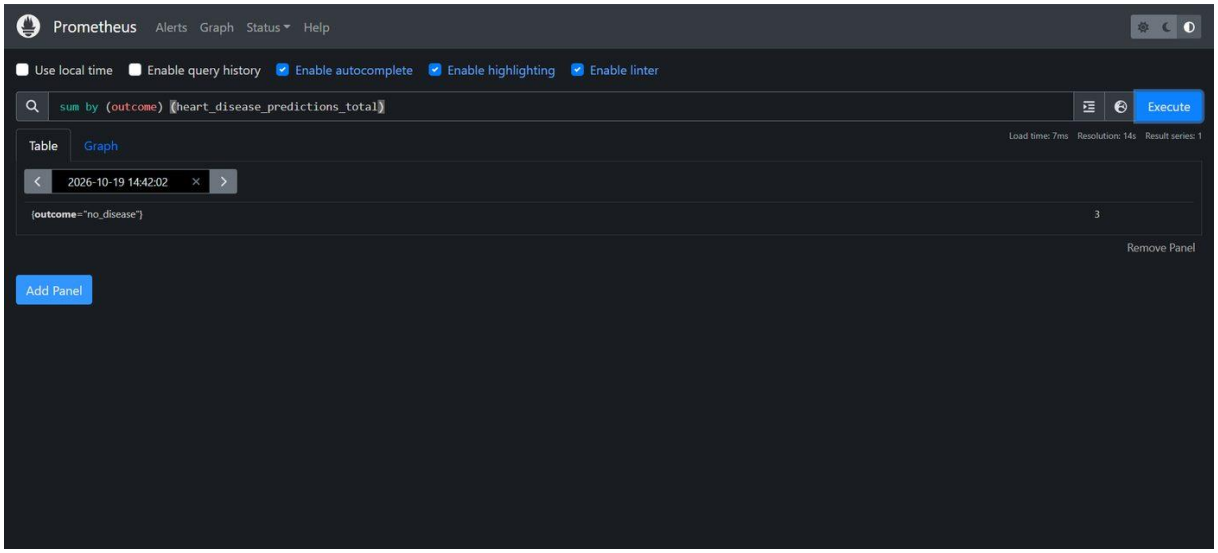


Figure 20: Prometheus query of predictions by outcome.

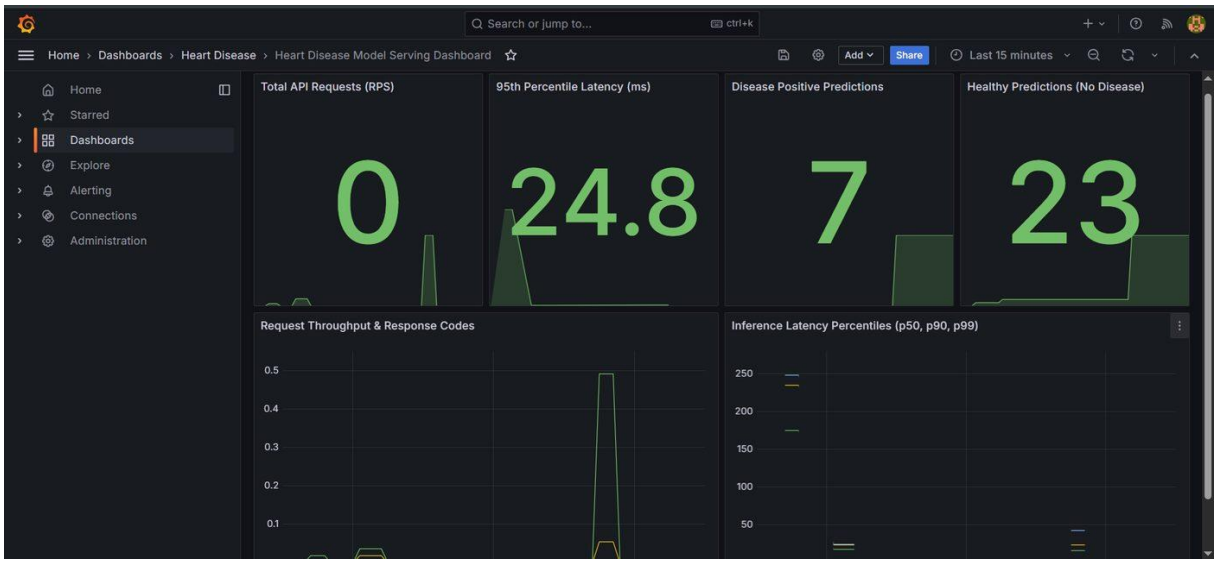


Figure 21: Grafana dashboard after sample traffic: p95 latency 24.8 ms, 7 disease-positive and 23 healthy predictions.

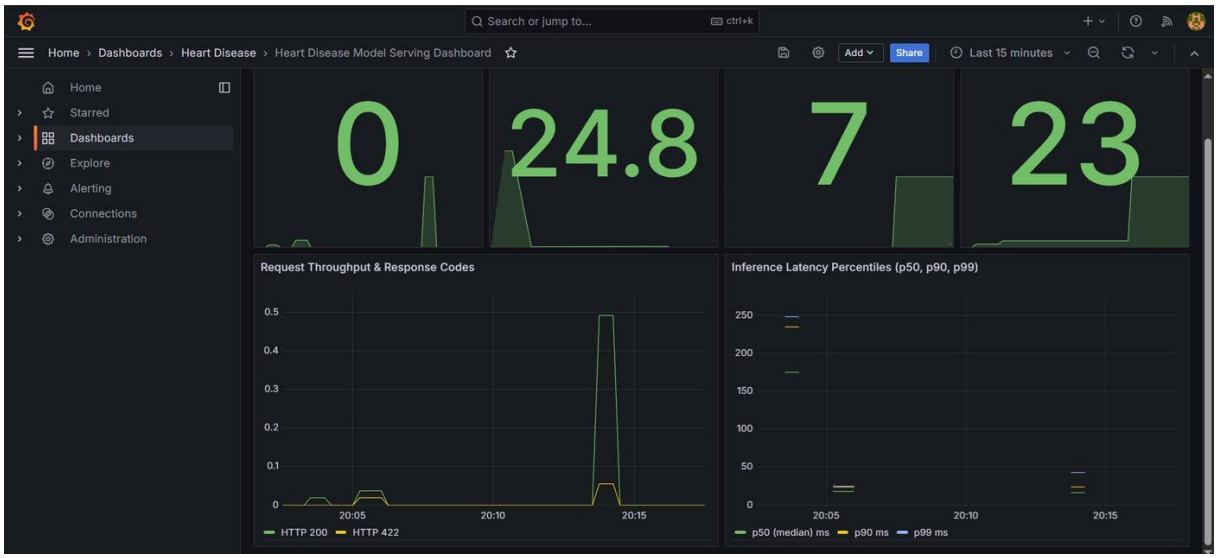


Figure 22: Grafana dashboard panels.

11. Architecture

The diagram shows the three stages of the system: data and training (tracked in MLflow), the CI/CD workflow that tests and verifies every change, and the serving stack with its observability layer.

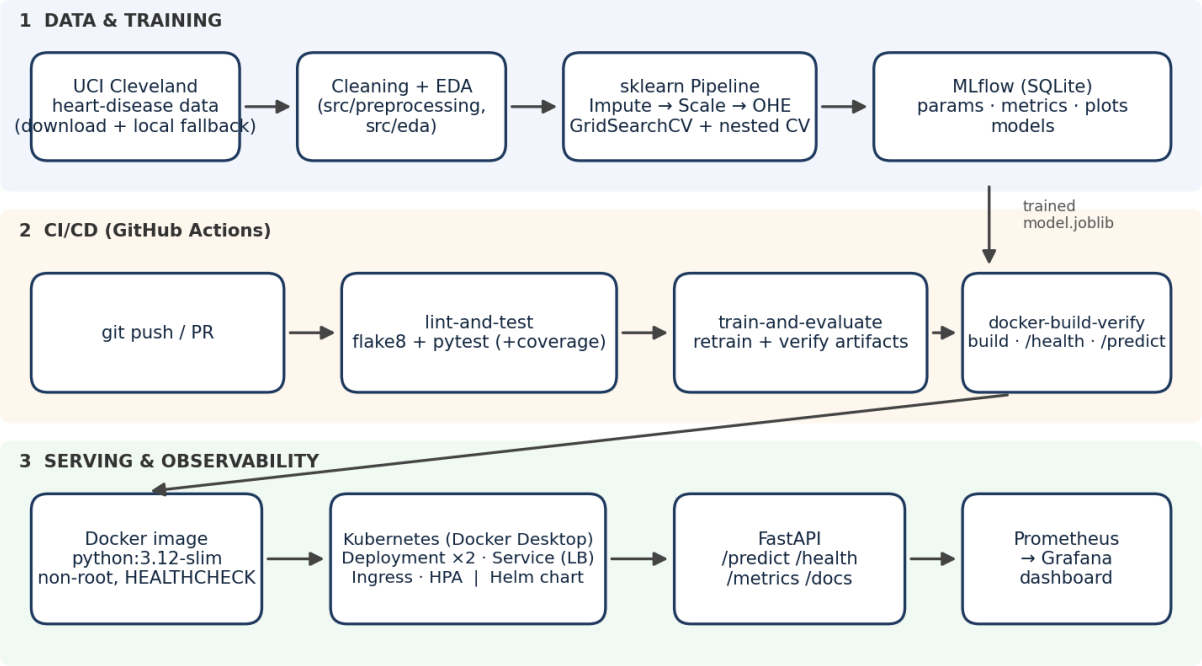


Figure 23: End-to-end architecture of the Heart Disease MLOps pipeline.

11.1 Repository layout

Path	Contents
src/	Data loading, preprocessing, EDA, training, evaluation, prediction and the FastAPI app
test/	Unit and API tests (pytest)
notebooks/	EDA, training/evaluation and inference notebooks
data/, dataset/	Raw and cleaned data; original UCI files
models/	Serialised pipeline and metadata
reports/	Figures, comparison tables, screenshots, report
k8s/, helm/	Kubernetes manifests and Helm chart
monitoring/	Docker Compose, Prometheus and Grafana provisioning
.github/workflows/ci.yml, Jenkinsfile	CI/CD definitions
scripts/	Serving test, traffic generator, MLflow and report helpers

12. Conclusion, Limitations and Future Work

The project delivers every item in the brief: a reproducible, leakage-safe modelling pipeline; MLflow tracking for all runs; a tested and linted codebase with a three-job GitHub Actions pipeline that fails on

code or test errors; a non-root Docker image verified in CI; Kubernetes and Helm deployment manifests; and request logging with a Prometheus/Grafana dashboard. The selected model, Logistic Regression, reaches 0.903 nested-CV ROC-AUC and 0.967 ROC-AUC on the held-out set with 0.929 recall.

Limitations

- The dataset is small (303 patients, single hospital), so test-set metrics are noisy and the model should not be regarded as clinically validated.
- Deployment is on a local Kubernetes cluster; HPA needs metrics-server for CPU-based autoscaling.
- The model is a decision-support prototype, not a diagnostic device.

Future work

- Deploy the same manifests to a managed cluster (GKE/EKS/AKS) with a public URL and TLS.
- Add data-drift and prediction-drift monitoring and alerting rules in Prometheus; automate retraining on drift.
- Register models in the MLflow Model Registry and add continuous delivery (image push and automated rollout) to the workflow.
- Add probability calibration and SHAP-based per-patient explanations to the API.

References and links

- Code repository: https://github.com/NIK183/heart-disease-mlops-pipeline
- UCI Machine Learning Repository, Heart Disease dataset (Cleveland): https://archive.ics.uci.edu/dataset/45/heart+disease
- **Demo video: https://drive.google.com/file/d/1F6ECuYfjuhEo-wyIA4QAP9B795hBEAIg/view?usp=sharing**